Video Ad Specialist

Video Ad Specialist Skill — Key Details

Skill Name

video-ad-specialist

Purpose

The Video Ad Specialist skill converts **marketing intent into structured short-form video ad scenes** optimized for social media platforms.

This skill **does not render the video itself**. Instead, it generates **structured JSON describing the ad strategy and scene sequence**, which is then consumed by the **Remotion rendering skill** to produce the final video.

This separates **marketing strategy generation from video rendering**.

Core Responsibilities

The Video Ad Specialist skill performs four primary functions.

1️⃣ Ad Strategy Generation

The skill determines the **best advertising structure** based on the product, audience, platform, and campaign goal.

Example strategy types:

- `product_showcase`
- `problem_solution`
- `testimonial`
- `limited_offer`

- `lifestyle`
- `meme_style`

The selected strategy determines:

- narrative structure
- pacing
- scene ordering
- messaging emphasis

2️⃣ Platform Optimization

The skill adapts ad pacing and structure depending on the target platform.

Example optimization logic:

Instagram Reels

Fast hook-driven structure:

Hook → Product → Benefit → CTA

Typical pacing:

Hook – ~2s
Product – ~5s
Benefit – ~3s
CTA – ~2s

Total: **10–12 seconds**

YouTube Shorts

More narrative structure:

Hook → Problem → Solution → CTA

Total: **12–15 seconds**

3️⃣ Scene Generation

The skill converts the chosen strategy into **structured scenes**.

Each scene represents a unit the Remotion renderer can translate into visuals.

Scene types may include:

- `hook`
- `problem`
- `product`
- `benefit`
- `testimonial`
- `offer`
- `cta`

Example scene structure:

```
{
 "type": "hook",
 "text": "Still drinking boring coffee?"
}
```

Example scene flow:

Scene 1 – Hook
Scene 2 – Product introduction
Scene 3 – Key benefit
Scene 4 – CTA

4️⃣ Remotion Configuration Output

The final output must be **JSON directly compatible with the Remotion rendering skill**.

Example output:

```
{
 "composition": "AdVideo",
 "props": {
   "style": "product_showcase",
   "duration": 12,
   "platform": "instagram_reels",
   "scenes": [
     { "type": "hook", "text": "Still drinking boring coffee?" },
     { "type": "product", "text": "Upgrade to Velocity Cold Brew" },
     { "type": "benefit", "text": "Smooth energy without the crash" },
     { "type": "cta", "text": "Try it today" }
   ]
 }
}
```

This JSON is passed directly into the **Remotion project**, which renders the final video.

Input Requirements

The skill expects marketing context such as:

Input	Example
Product	Cold Brew Coffee
Target Audience	Busy professionals
Platform	Instagram Reels
Campaign Goal	Drive purchases

If inputs are missing, the skill should infer reasonable defaults.

Output Format

The skill must output **valid JSON only**.

Required fields:

composition
props.style
props.duration
props.platform

props.scenes

Each scene must include:

type
text

Optional properties may include:

visual
transition
animation

Relationship to Other Skills

This skill sits in the pipeline **before Remotion rendering**.

Pipeline flow:

User Prompt
↓
Video Ads Agent
↓
Video Ad Specialist Skill
↓
Ad Scene JSON
↓
Remotion Rendering Skill
↓
Rendered Video

The Video Ad Specialist **generates the ad structure**, while the Remotion skill **renders the final video asset**.

Project Role in the Marketing Pipeline

Within the full AI marketing system:

Research Agent
↓
Ad Creative Designer
↓
Video Ad Specialist
↓
Remotion Rendering
↓

Copywriter Agent
↓
Distribution Agent

The Video Ad Specialist acts as the **bridge between marketing strategy and video production**.

Design Principle

The output format should be **reusable across multiple marketing assets**.

The same ad strategy JSON can potentially drive:

- Remotion video ads
- Static image ads
- Social captions
- Distribution metadata

This ensures **consistent messaging across media formats**.

Ad Creative Designer

1. Concept

Instead of generating pixels with an AI model, the agent generates a **design specification**.

The frontend reads that specification and **renders the ad**.

So the pipeline becomes:

User Prompt
↓
Image Ads Agent
↓
Ad Creative Designer Skill
↓
Layout JSON

↓
Frontend Renderer
↓
Export PNG

The AI creates the **design blueprint**, not the image itself.

---

2. What the Agent Generates

The agent outputs structured layout data.

Example:

```
{
 "format": "instagram_square",
 "background": "#f5f1e8",
 "elements": [
   {
     "type": "headline",
     "text": "Upgrade Your Morning",
     "x": 80,
     "y": 120,
     "fontSize": 64
   },
   {
     "type": "subtext",
     "text": "Smooth Cold Brew Energy",
     "x": 80,
     "y": 200
   },
   {
     "type": "cta",
     "text": "Shop Now",
     "x": 80,
     "y": 350
   },
   {
     "type": "image",
     "src": "/assets/coffee.png",
     "x": 600,
     "y": 200,
     "width": 300
   }
 ]
}
```

That JSON becomes your **design spec**.

---

3. The Frontend Renderer

You build a simple frontend using **React or Next.js**.

It renders the layout onto an **HTML5 Canvas**.

Basic idea:

```
const canvas = document.getElementById("adCanvas");
const ctx = canvas.getContext("2d");

ctx.fillStyle = layout.background;
ctx.fillRect(0,0,canvas.width,canvas.height);
```

Then loop through elements.

---

4. Rendering Text

Example headline rendering:

```
ctx.font = "bold 64px Inter";
ctx.fillStyle = "#111";
ctx.fillText("Upgrade Your Morning", 80, 120);
```

The agent controls:

text
position
font size

---

5. Rendering Images

Example:

```
const img = new Image();
img.src = "/assets/coffee.png";

img.onload = () => {
 ctx.drawImage(img, 600, 200, 300, 300);
};
```

This lets you place product photos.

---

6. Rendering Buttons / CTA

Simple rectangle:

```
ctx.fillStyle = "#111";
ctx.fillRect(80, 320, 220, 60);

ctx.fillStyle = "#fff";
ctx.fillText("Shop Now", 110, 360);
```

Your agent decides:

button text
position
style

---

7. Exporting the Image

Once the canvas is rendered:

```
const png = canvas.toDataURL("image/png");
```

Download:

```
const link = document.createElement("a");
link.download = "ad.png";
link.href = png;
link.click();
```

Boom — **generated ad image**.

---

8. Formats for Different Platforms

Your agent should generate different sizes.

Examples:

Instagram Post
1080 x 1080
Instagram Story

1080 x 1920
YouTube Thumbnail
1280 x 720

The JSON could specify:

```
{
"width":1080,
"height":1080
}
```

The canvas resizes accordingly.

---

9. What Your Custom Skill Does

Your **Ad Creative Designer skill** generates the layout spec.

Input:

Product
Audience
Platform
Style

Output:

layout JSON

This is where the **AI creativity happens**.

---

10. Folder Structure

Clean structure for the project.

```
image-agent/

agents/
  image_ads_agent.md

skills/
  ad-creative-designer/

frontend/
  components/
    CanvasRenderer.tsx
```

```
  pages/
    generator.tsx

outputs/
  ads/
```

---

11. Example Workflow

User input:

Product: Cold Brew Coffee
Platform: Instagram
Style: Minimal

Agent generates layout:

headline
product image
CTA
background

Frontend renders:

ad_instagram.png

---

12. Why This Approach Is Great for Your Project

This method is perfect because:

✔ No paid APIs
✔ No GPU needed
✔ Fully explainable in tutorial
✔ Deterministic results
✔ Easy to customize

And it still feels like **AI-generated design**.

---

13. One Trick That Makes This Much Better

Instead of letting the AI randomly place things, define **layout templates**.

Example:

template: product_left

template: product_right
template: centered_minimal

The agent chooses the template.

This prevents **ugly layouts**.

Ad Creative Designer Modification

Update the existing `ad-creative-designer` skill to extend its capabilities.

Do not remove or rewrite the existing JSON layout generation logic. The design JSON must still be produced exactly as defined in the skill.

Add the following **additional step after the JSON output**.

Step 7: HTML Ad Rendering

After generating the design JSON, convert the layout into a rendered HTML advertisement.

Generate the following files:

- `ad.html`
- `styles.css`

The HTML must represent the layout defined in the JSON.

Requirements:

• Canvas rendering is no longer the primary output method
• The HTML layout must match the selected layout type (`product_focus`, `split`, `lifestyle`)
• The design must render at **1080x1080** for Instagram square format
• Typography must clearly prioritize headline > subtext > CTA
• The CTA must be visually distinct (button style)

Example HTML structure:

```
<div class="ad-container">

  <div class="headline">Upgrade Your Morning</div>
```

```
  <div class="subtext">Smooth cold brew energy with zero bitterness</div>

  <img class="product" src="coffee_can.png" />

  <button class="cta">Shop Now</button>

</div>
```

CSS must enforce:

- `width:1080px`
- `height:1080px`
- balanced spacing
- clean modern marketing layout

Step 8: Playwright Screenshot Rendering

After generating the HTML layout, render it using **Playwright**.

Process:

1. Launch Chromium using Playwright
2. Set viewport to `1080x1080`
3. Load the generated `ad.html`
4. Wait for images to render
5. Capture a screenshot
6. Save the image as `instagram_ad.png`

The screenshot must capture the **full 1080x1080 ad frame**.

Step 9: Output Storage Rules

All generated files must follow the project output rules.

Create a new task folder using:

outputs/TASKNAME_DATE/

Inside the task folder create:

outputs/TASKNAME_DATE/ads/

Save files here:

ad.html

styles.css

instagram_ad.png

Do not place any generated files outside the `outputs` directory.

Resulting Outputs

The skill now produces three deliverables:

1. Design JSON (existing behavior)
2. HTML ad layout
3. Final rendered Instagram image via Playwright screenshot

Marketing Research Agent

The Research Agent takes a **business niche or topic** as input and generates **structured marketing intelligence** that the rest of the pipeline can consume.

It uses **Tavily web research** to analyze market trends, competitor messaging, and common audience interests. Instead of returning a simple list of ideas, the agent organizes the research into **structured fields** such as:

- content topics
- marketing angles
- keywords
- video concepts
- ad hooks

The important part is that the output is **machine-readable structured data**, not just text. That means downstream agents — like the video generator, visual ads agent, and copywriter — can **directly consume this research as input**.

So in the pipeline, the Research Agent acts as the **strategic layer**, producing the campaign

insights that drive all the creative assets generated afterward.

Copywriter Agent

Copywriter Agent — Operational Rules

Rule 1 — Always Reference Knowledge Files

Before generating any copy, the agent must review relevant files inside the `knowledge/` folder.

These files define:

brand voice
tone
product messaging
brand guidelines

Typical files may include:

knowledge/brand_identity.md
knowledge/product_campaign.md
knowledge/platform_guidelines.md

The agent must align all generated copy with:

- brand voice
- product positioning
- CTA style
- messaging priorities

If there is a conflict between generic copy and brand guidelines, **brand guidelines take priority**.

Rule 2 — Reference Research Outputs When Available

If the prompt specifies a research output or campaign run, the agent should reference relevant files in the `outputs/` folder.

Example structure:

```
outputs/
  campaign_coldbrew_demo/
    research/
      research_output.json
```

The agent should extract from the research output:

content_topics
content_angles
keywords
ad_hooks
video_ideas

These should influence:

- messaging hooks
- platform copy
- hashtags
- YouTube tags

This ensures the copy **aligns with the campaign strategy produced earlier**.

Rule 3 — Campaign Consistency

All generated copy must use **a single campaign angle** selected from the research output.

Example:

Campaign Angle: Upgrade Your Morning

That angle should remain consistent across:

Threads post
Instagram caption
YouTube title

Rule 4 — Platform Adaptation

Each platform must follow its **native style**.

Platform	Style

Threads	short, conversational
Instagram	storytelling + CTA
YouTube	SEO optimized

Never generate identical copy across platforms.

Rule 5 — Follow Platform Formatting Guidelines

Reference `knowledge/platform_guidelines.md`.

Example constraints:

Instagram:

max 3 emojis
3–5 hashtags
clear CTA

Threads:

short form
1–2 short paragraphs

YouTube:

title optimized for search
description includes keywords
tags list included

Rule 6 — Output Must Be Structured

Return results as structured output so other agents can consume them.

Example format:

```
{
"threads_post": "...",

"instagram_caption": "...",

"youtube": {
 "title": "...",
 "description": "...",
 "tags": []
}
```

}

Rule 7 — Output File Location

All outputs must be saved following the pipeline structure:

outputs/{task_name}_{date}/copy/

Example:

outputs/coldbrew_campaign_2026-03-15/copy/

Files:

threads_post.txt
instagram_caption.txt
youtube_metadata.json

Why These Rules Matter

These rules guarantee:

✔ brand alignment
✔ campaign consistency
✔ integration with previous agents
✔ predictable outputs

It also makes the agent behave like a **real marketing team member**, not a generic text generator

Distribution Agent

Distribution Agent — Key Details

Purpose:
Handles media hosting (Supabase), publishing readiness, and post scheduling advisory. References outputs from Research Agent and Copywriting Agent to construct metadata and scheduling recommendations. Publishes only when the user explicitly references the generated MD file.

Responsibilities

1. **Supabase Media Hosting**

- Uploads all outputs from `outputs/<task_name>/<date>/` to Supabase (`campaign-uploads` bucket).
- Generates **public URLs** for Instagram (images/videos).
- Ensures filenames are unique per task.

2. **Publishing Layer**
   - Prepares metadata for Instagram (caption, hashtags, CTA) and YouTube (title, description, tags).
   - Only executes posting **when user explicitly references the Publish MD file**.
   - Can mock YouTube posting if OAuth not configured.
3. **Post Scheduling Advisory**
   - Generates an MD file per task:
     - `Publish <task_name> <YYYY-MM-DD>.md`
     - Includes media URLs, recommended posting times, metadata summary, and notes.
   - Uses **knowledge files** (`brand_identity.md`, `platform_guidelines.md`) and **Research outputs** for scheduling guidance.
4. **Pipeline Awareness**
   - References upstream agents for structured outputs:
     - Research Agent → trends, topics, keywords
     - Copywriting Agent → captions, titles, tags
   - Knows output folder structure to locate latest media and metadata.

Input Requirements

Input	Example / Source
Task Name	`coldbrew_campaign`
Task Date	`2026-03-15`

Media Files	`outputs/coldbrew_campaign_2026-03-15/video1.mp4`, `ad1.png`
Research JSON	`outputs/coldbrew_campaign_2026-03-15/research.json`
Copywriting JSON	`outputs/coldbrew_campaign_2026-03-15/copy.json`
Publish Command	User cites `Publish coldbrew_campaign 2026-03-15.md`

Output

- **MD File:** `Publish <task_name> <date>.md`
    - Media file URLs (Supabase)
    - Suggested posting times per platform
    - Metadata summary (captions, hashtags, titles, descriptions)
    - Task completion notes and scheduling advice
- **Execution Rule:** Publishing happens **only when user explicitly confirms using this MD file**.

Rules / Constraints

1. Must reference **knowledge files** and relevant **research outputs** before generating schedules or metadata.
2. All media must be **uploaded to Supabase** with valid public URLs.
3. No posting without explicit user approval.
4. Maintain consistent output folder structure: `outputs/<task_name>/<date>/`
5. Supports **images and videos**; multiple files per task allowed.

Optional Extensions

- Integration with BullMQ + Redis for automated queueing and scheduled execution.

- Direct API posting for YouTube with OAuth.
- Multi-platform expansion (Threads, TikTok, LinkedIn)

Orchestrator Agent

You are the AI pipeline orchestrator using BullMQ + Redis. Tasks (Jobs) are enqueued and executed asynchronously. Each Job Payload represents a campaign task, containing:

- task_name
- task_date
- source_folder (optional, for skipping research)
- user_flags (skip_research, skip_image, skip_video)
- platform_targets (Instagram, YouTube)

Instructions:

1. Receive a Job Payload from the Orchestrator Skill.
2. Check dependencies for each agent in the pipeline:
   - Research Agent → must run first unless skip_research is true.
     - If skipped, confirm the user has uploaded the source folder inside `assets/<task_name>/`.
     - If missing, return a note: "Task cannot proceed until source folder is uploaded."
   - Ad Creative Designer → runs after research or after skip confirmation.
   - Video Ad Specialist → runs after research or after skip confirmation.
   - Copywriter Agent → runs after research or skip confirmation.
   - Distribution Agent → runs last, after all outputs are ready.
3. Enqueue jobs in BullMQ with proper dependencies.
4. Track completion status for each job.
5. Support optional skips for image/video creation (ad designer or video specialist). If skipped, mark job as complete without executing.
6. Generate logs and summaries for the Orchestrator agent to report.
7. Notify when pipeline completes successfully or if errors occur.

Output example per job:

```
{
  "job_name": "video_ad_specialist",
  "status": "queued/running/complete/failed",
  "dependencies": ["research_agent"],
  "notes": "Skipped per user flag" // if applicable
}
```

Orchestrator Skill — Key Details

Purpose:
Receives a **Job Payload** and runs the AI content pipeline as a single coordinated workflow. Manages agent dependencies, optional skips, and validation of uploaded assets. Tracks job status and ensures outputs are ready for Distribution Agent.

Responsibilities

1. **Job Payload Intake**
    - Receives all task parameters:
        - `task_name`, `task_date`
        - `source_folder` (for research skip verification)
        - User flags: `skip_research`, `skip_image`, `skip_video`
        - `platform_targets` (Instagram, YouTube)
    - Validates payload structure.
2. **Pipeline Execution**
    - Default pipeline order:
        - Research Agent → generates structured marketing insights.
        - Ad Creative Designer → generates static ad layouts (JSON).
        - Video Ad Specialist → generates videos using Remotion + custom skill.
        - Copywriter Agent → generates captions, titles, tags.
        - Distribution Agent → uploads outputs, generates Publish MD, advises schedule.
    - Optional skips:
        - `skip_research` → verify `assets/<task_name>` exists; if not, block pipeline.
        - `skip_image` or `skip_video` → job marked complete, no generation occurs.
3. **Dependency Management**

- Ensures each agent executes only after required upstream outputs exist (or skip confirmed).
- Uses **BullMQ + Redis** to enqueue jobs and track completion.
- Supports parallel execution for independent tasks (e.g., video and image jobs can run simultaneously after research).

4. **Job Tracking**
   - Each agent's execution is treated as a **Job**.
   - Job states: `queued`, `running`, `complete`, `failed`.
   - Logs generated for every job to help debug or show pipeline progress.
5. **User Control**
   - Pipeline **does not automatically publish content**.
   - Publishing only occurs via Distribution Agent **when user references the Publish MD file**.

Input Requirements

Input	Example / Notes
Job Payload	JSON containing task_name, task_date, source_folder, skip flags, platform_targets
Assets Folder	`assets/<task_name>/` (required if skipping research)
Knowledge Files	`brand_identity.md`, `platform_guidelines.md`, `product_campaign.md`
Research Output	Optional, used if research skipped
Copywriting Output	Optional, used by Distribution Agent

Outputs

- Job logs per agent (status, notes)

- Final pipeline completion report
- Distribution-ready Publish MD files in `outputs/<task_name>/<date>/`
- Optional warnings if skips cannot be validated (missing source folder)

Rules / Constraints

1. **Pipeline must respect agent dependencies**.
2. **Research cannot be skipped** without validated source folder.
3. Image and video jobs can be skipped at user discretion.
4. All outputs must follow `outputs/<task_name>/<date>/` folder convention.
5. Publishing is **manual**: only triggered via Distribution Agent with MD file reference.

Optional Level-Up Features

- Add **time-based scheduling** using BullMQ delayed jobs.
- Retry logic for failed jobs automatically.
- Add multi-platform posting (Threads, TikTok, LinkedIn) when APIs available.

This way, you can now **run the full pipeline from a single Job Payload**:

1. User submits a JSON payload → Orchestrator Skill triggers
2. Jobs are enqueued in BullMQ + Redis → run according to dependencies
3. All agents execute automatically or skip per flags
4. Distribution Agent is ready to publish when user confirms